Orchestration de conteneurs : Kubernetes et Docker Swarm

Docker (ou Compose) sait lancer des conteneurs sur **une** machine. Dès qu'il faut plusieurs machines, de la haute disponibilité, des mises à jour sans coupure ou de l'autoréparation, il faut un **orchestrateur** : on lui décrit l'état voulu (« 3 copies de cette image, exposées sur le port 80 ») et il fait en sorte que la réalité y reste conforme. Cette fiche distingue les concepts généraux, puis Kubernetes (K8s) et Docker Swarm.

> Les sections « Swarm » et les notions de Kubernetes absentes du dépôt sont de la **théorie non vérifiée dans ce dépôt**. Seuls les manifestes de `k8s/` sont ancrés sur le code réel.

Ce que fait un orchestrateur

Besoin	Réponse de l'orchestrateur
Placer les conteneurs	Un ordonnanceur (scheduler) choisit le nœud selon les ressources disponibles
Rester en vie	Un conteneur qui tombe est recréé : on parle d'autoréparation (self-healing)
Passer à l'échelle	On change le nombre de réplicas ; le trafic est réparti entre elles
Découvrir les services	Un nom stable (DNS, Domain Name System) pointe vers les réplicas, quelle que soit leur adresse
Mettre à jour	Mise à jour progressive (rolling update) : les anciennes réplicas sont remplacées une à une
Configurer	Variables d'environnement, secrets et configs injectés, sans les mettre dans l'image

L'idée centrale est le **modèle déclaratif** : on ne dit pas « démarre ce conteneur » (impératif), on décrit l'état souhaité dans un fichier YAML, et une boucle de contrôle compare en continu l'état réel à l'état voulu pour corriger l'écart.

Kubernetes (K8s)

Kubernetes est l'orchestrateur le plus répandu. « K8s » est une abréviation : K, 8 lettres, s.

Objets à connaître :

- **Pod** : la plus petite unité déployable, un ou plusieurs conteneurs qui partagent réseau et stockage. Un Pod est éphémère : on ne le crée presque jamais à la main.
- **Deployment** : décrit un modèle de Pod et le nombre de réplicas voulu ; il gère la mise à jour progressive et le retour arrière (rollback).
- **Service** : une adresse stable devant un groupe de Pods, sélectionnés par leurs **labels**.
- **Secret / ConfigMap** : configuration injectée dans les Pods (les secrets ne sont qu'encodés en base64 par défaut, pas chiffrés).
- **Namespace** : cloisonnement logique au sein d'un cluster.

Les types de Service :

Type	Visible depuis	Usage
`ClusterIP` (défaut)	l'intérieur du cluster	communication entre services
`NodePort`	un port de chaque nœud	tests, accès simple
`LoadBalancer`	l'extérieur, via l'équilibreur du fournisseur cloud	exposer une application

Architecture : un **plan de contrôle** (API server, ordonnanceur, contrôleurs, base `etcd` qui stocke l'état) pilote des **nœuds** (worker) où un agent, le `kubelet`, lance les Pods. On interagit avec `kubectl`, par exemple `kubectl apply -f k8s/`.

Docker Swarm

Swarm est l'orchestrateur intégré à Docker : `docker swarm init` transforme une machine en cluster, sans rien installer de plus.

- Des **managers** (qui décident, et se synchronisent avec l'algorithme de consensus Raft) et des **workers** (qui exécutent).
- Un **service** : l'équivalent approximatif d'un Deployment, avec un nombre de réplicas.
- On déploie un fichier Compose avec `docker stack deploy -c docker-compose.yml <nom>`. La clé `deploy:` (replicas, update_config, resources) n'a de sens que dans ce mode.
- Le réseau **overlay** relie les conteneurs de plusieurs machines, et le **routing mesh** rend un port publié accessible sur n'importe quel nœud.
- `docker stack deploy` **ignore** `build:` : l'image doit déjà exister dans un registre.

Kubernetes face à Swarm

Critère	Kubernetes	Docker Swarm
Mise en place	Plus lourde (cluster, réseau, ingress)	Une commande : `docker swarm init`
Format	Manifestes YAML propres à K8s	Fichier Compose étendu (`deploy:`)
Fonctionnalités	Très riches (autoscaling, opérateurs, RBAC, ingress)	Plus limitées
Écosystème	Énorme, standard de fait, offres cloud gérées	Plus réduit
Courbe d'apprentissage	Raide	Douce si on connaît Compose
Cas d'usage typique	Production à grande échelle	Petits clusters, besoins simples

Dans ce dépôt

Docker Compose (docker-compose.yml) est l'étape *mono-machine* : il construit l'image et l'expose sur le port 80 de l'hôte. Kubernetes est l'étape suivante, décrite par deux manifestes :

- deployment-app.yaml:6 : `replicas: 1`. Le Deployment crée un Pod à partir du modèle `template`, retrouvé grâce au label `app: olympic-games-app` (lignes 7-9 pour `selector`, 11-13 pour les labels du modèle). Le sélecteur doit correspondre aux labels du Pod, sinon le Deployment est rejeté.
- service-app.yaml:6-7 : le `selector` du Service reprend le même label, c'est ce lien qui relie le Service aux Pods.
- service-app.yaml:12 : `type: LoadBalancer`, pour une exposition à l'extérieur du cluster.
- deployment-app.yaml:17 et :20 : deux `TODO`, l'image à tirer du registre GitLab et les `imagePullSecrets` pour autoriser Kubernetes à tirer une image d'un registre privé. Le gabarit values.tpl.yaml prévoit les variables `image`, `port` et `imagePullSecret` à remplir.
- **Incohérence à corriger** : le manifeste déclare toujours le port 80 (deployment-app.yaml:19, service-app.yaml:10-11), alors que l'image tourne maintenant sans root et écoute sur **8080** (voir Dockerfile). Il faut passer `containerPort` et `targetPort` à 8080.
- Rien dans le dépôt n'utilise Swarm.

À retenir

1. Un orchestrateur applique un **état déclaré** (YAML) et le maintient par une boucle de contrôle : c'est ce qui donne l'autoréparation, la mise à l'échelle et les mises à jour progressives.
2. Dans Kubernetes, le **Deployment** gère les Pods et le **Service** leur donne une adresse stable ; les deux sont liés par des **labels**, pas par des noms.
3. Swarm est plus simple (une commande, format Compose) mais moins riche ; Kubernetes est le standard pour la production.
4. Compose, Swarm et Kubernetes forment une progression : une machine, un petit cluster, un cluster à grande échelle. Ici, Compose est abouti et Kubernetes est un squelette à compléter (image, registre privé, port 8080).